RedlineDB: A Rust-Native, Concurrent-Write Embedded SQL Engine That Stays SQLite-Compatible Without Inheriting Its Concurrency Cliff

Anonymous Submission
RedlineDB Authors
Email: redlinedb@example.org

***Abstract*—SQLite is the most-deployed database engine on Earth, but its single-writer write-ahead log imposes a hard concurrency ceiling that the rest of the data tier has long since outgrown. We rebuilt SQLite's API contract from scratch in safe Rust and measured what it costs to keep the contract while removing the cliff. RedlineDB exposes the `sqlite3.h` C ABI and a native `rldb_*` surface from roughly 35 KLOC of safe Rust active source—approximately one-seventh of the SQLite amalgamation's roughly 250 KLOC of C—and replaces the single-writer WAL with a multi-version concurrency control kernel, B-link-style index lifecycle, and a group-commit WAL coordinator. On a 128-vCPU host we ran 1,280 measured benchmark child runs across 8 workloads, 2 durability modes, 8 thread levels, and 5 repetitions per cell, against SQLite 3.x bundled by `rusqlite`. RedlineDB delivers 7.9–8.3× throughput on mixed and writers-disjoint OLTP at 64 threads, holds parity at the 64-thread point-read peak (0.99×), and trails on two contention-bound workloads (hot-row update at 0.21×, secondary-index range scan at 0.012×) which we discuss honestly. A deterministic crash oracle observed 0 lost-acked-commits across 24 fail-point scenarios; the recovery matrix passed 36/36 cases and the SQL compatibility matrix passed 40/40. We contribute (i) a safe-Rust MVCC kernel with B-link concurrent indexing, (ii) a SQLite-compatible C ABI shim, (iii) a deterministic crash certification protocol, (iv) a reproducible bin-packed parallel benchmark harness, and (v) the resulting empirical scaling profile.**

***Index Terms*—embedded database, MVCC, concurrent indexing, Rust, SQLite compatibility, WAL, deterministic testing, benchmark**

I. Introduction

SQLite is the most widely deployed database engine on Earth [1]: every major mobile platform, every mainstream web browser, the in-flight entertainment system on commercial aircraft, and a long tail of embedded controllers ship a copy. Its durability story is conservative and well-documented [2], its on-disk format has been stable since 2004, and its test discipline—roughly 92.6 million lines of test code against the amalgamation [3]—is unmatched among open-source relational engines.

The cost of that conservatism is concentrated in one place: the write-ahead log. SQLite's WAL is a single-writer log [4]; any number of readers may proceed concurrently against the most recent committed snapshot, but at most one writer may hold the database lock at a time. On modern many-core hosts this serializes every write-bearing transaction onto a single critical section regardless of whether two transactions touch disjoint rows. Workloads that mix even a small fraction of writes flatten well below the host's available parallelism. The architecture of the engine—a hand-rolled five-class type affinity system, a bespoke 250-KLOC C amalgamation [5], a virtual-machine bytecode whose opcode set has accreted for two decades—also makes incremental modernization difficult: any patch that touches the WAL contract risks an ecosystem-scale regression.

We argue these two cliffs (concurrency ceiling and engineering surface) can be removed without giving up the SQLite *contract*—the C API, the SQL dialect, the file-per-database mental model, the embedded single-process deployment. RedlineDB is a Rust-native, MVCC, concurrent-write embedded SQL engine that is API-compatible with SQLite. The kernel, parser, planner, executor, public Rust facade, and benchmark harness together total approximately 35 KLOC of safe Rust active source (Table I); only the FFI shim contains `unsafe` blocks, and those exist because the C ABI requires them. The kernel achieves ARIES-style recovery [6] on top of a slotted-heap MVCC store, and the index layer follows the B-link discipline [7], [8] so that concurrent writers on disjoint key ranges do not block each other.

Contributions.

1) A safe-Rust MVCC kernel with B-link-style concurrent index lifecycle, group-commit WAL, and ARIES-style redo/undo recovery—packaged as an embedded library with the SQLite mental model intact.
2) A SQLite-compatible C ABI shim path (`sqlite3.h` ∪ `rldb_*`) that allows existing SQLite clients to link against RedlineDB without source changes for the covered surface.
3) A deterministic crash certification protocol: 16 failpoint sites exercised under a fsynced-ack child-oracle harness, observing 0 lost acked commits across 24 scenarios.
4) A reproducible bin-packed parallel benchmark harness with real-warmup discard, recursive on-disk footprint accounting, and strace-passthrough telemetry, packaged

into a single manifest-stamped certification run.
5) An empirical scaling profile: 7.9–8.3× throughput on mixed and writers-disjoint OLTP at 64 threads against SQLite 3.x bundled by `rusqlite` [9], parity at 64-thread point reads, and a transparent accounting of the two workloads where SQLite still wins.

The rest of this paper is organized as follows. Section II discusses related work; Section III presents the system architecture; Section IV covers implementation; Section V describes our methodology; Section VI reports the measured results; Section VII is the honest postmortem; Section VIII concludes.

II. Background and Related Work

A. The SQLite WAL Contract

SQLite's write-ahead log [4] provides snapshot reads with a single-writer constraint. A new connection acquires the appropriate shared or exclusive lock; readers see the most recent committed checkpoint; the writer appends frames to the WAL file and commits by writing a frame whose header marks end-of-transaction. The contract is robust and easy to reason about, but it serializes every writer through one lock. On a 64-thread workload mixing reads and writes, throughput plateaus at the rate at which a single writer can fsync.

B. ARIES, B-link Trees, and the Index Concurrency Problem

ARIES [6] established the canonical pattern of write-ahead logging with redo/undo records, a checkpoint that does not require quiescing the database, and physiological logging that combines page-level redo with logical undo. RedlineDB's WAL coordinator follows this template.

For the index layer, the obvious analogue of "serialize one writer" is to lock the entire B-tree at the root. Lehman and Yao showed in 1981 [7] that B-link trees can support concurrent inserts, splits, and lookups by augmenting each internal node with a right-link pointer and following the link when a search overshoots a recently split node. Srinivasan and Carey [8] empirically validated that this discipline outperforms lock-coupling on multi-core hardware. Graefe [10] surveys the broader space of B-tree locking strategies. Bayer and McCreight [11] are the original B-tree reference. We adopt the B-link mental model: index leaves carry transactional mark/unmark operations rather than physical deletions during a live transaction, which lets the executor undo a partially-applied DML without quiescing the index.

C. MVCC and Snapshot Isolation

Multi-version concurrency control [12] decouples readers from writers by keeping multiple versions of a row, indexed by commit timestamps. Reed's 1978 thesis [13] introduced the per-version naming scheme that all modern MVCC engines descend from. Berenson et al. [14] cataloged the anomalies—write skew, read-only-anomaly, lost update—that show up between read committed and serializable, and Cahill et al. [15] introduced serializable snapshot isolation (SSI) as a runtime discipline that detects dangerous structures. Kung and Robinson [16] are the canonical reference for optimistic concurrency control.

RedlineDB currently implements snapshot isolation: a transaction sees all rows committed at or before its read-CSN, and writes acquire row locks rather than running a certification phase. SSI is future work (Section VII). In practice, SI has been the production isolation level of PostgreSQL [17], [18], InnoDB [19], and Hekaton [20] for years; we follow the same convention. Gray's classic transaction-concept paper [21] and the granularity-of-locks report [22] frame why row-granularity locks matter for the workloads where MVCC alone is not enough.

D. Embedded Databases and Where RedlineDB Sits

LMDB [23] is the canonical memory-mapped embedded store: single-writer, copy-on-write B^+-tree, no SQL. LevelDB [24] and RocksDB [25] are LSM-based key-value stores; they are write-throughput optimized but expose no relational surface. In the Rust ecosystem, sled [26] (now in maintenance mode) and redb [27] are pure-Rust key-value stores; neither speaks SQL or the SQLite ABI. libSQL [28] is Turso's open-contribution fork of SQLite written in C and Rust; it preserves the SQLite WAL model and its single-writer constraint by construction. DuckDB [29] is the embedded analytical comparison point, but its design center is column-store OLAP, not OLTP. HyPer [30] and the fast serializable MVCC follow-up [31] are closer in spirit—OLTP, MVCC, in-memory—but live in a server context. RedlineDB occupies the cell that none of these fill: embedded, SQL, MVCC, multi-writer, SQLite-ABI-compatible.

E. Determinism, Failpoints, and Crash Testing

Failpoint discipline—compile-neutral injection points wired to a text-driven schedule—was popularized by TiKV's `fail-rs` [32]. SQLite's own testing program [3] and the SQLLogicTest harness [33] set the durability bar; Jepsen reports contextualize what storage engines can and cannot promise [34]. FoundationDB [35] and TigerBeetle [36] have shown how deterministic simulation lets a small team certify a transactional kernel against adversarial schedules; whitebox fuzzing [37] and property testing [38], [39] round out the modern toolkit. RedlineDB's certification protocol (Section V) draws from these lineages. The Rust safety story [40], [41] and the formalization of "safe outside `unsafe`" [42] are what make a deterministic kernel feasible without C-style memory hazards. Bw-trees [43] and consensus protocols [44] are noted here as adjacent design points we did not adopt.

III. System Architecture

Figure 1 shows the layered architecture. The repository splits into five Cargo crates: `redlinedb-kernel` (catalog, engine, index, page heap, WAL, storage), `redlinedb-sql` (parser, planner, executor), `redlinedb` (the public Rust facade and registry), `redlinedb-ffi` (C ABI surface), and `redlinedb-bench` (the certification harness). Crate

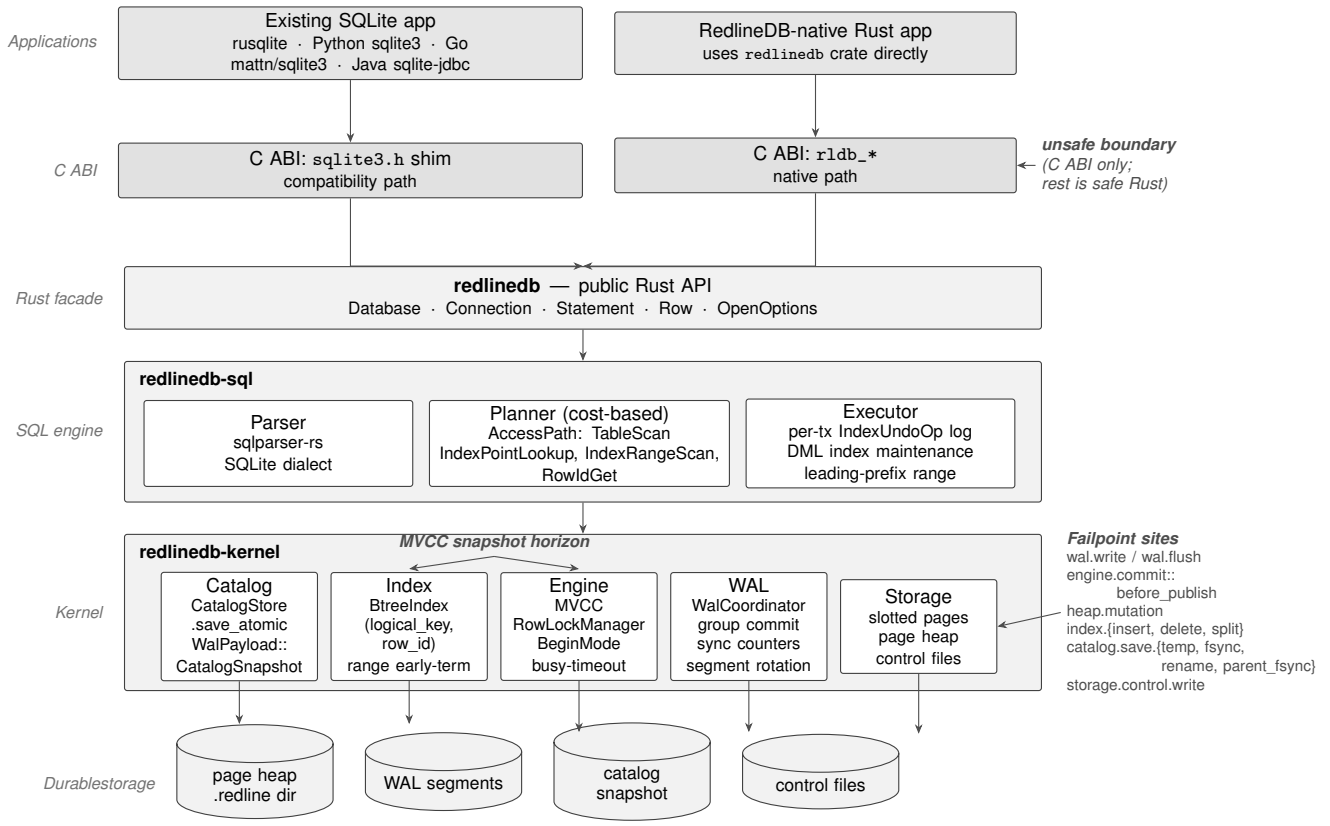

Fig. 1. RedlineDB layered architecture. The five horizontal bands are the C ABI shim (`sqlite3.h`), the native `rldb_*` FFI, the public Rust facade (`redlinedb`), the SQL layer (parser/planner/executor), and the kernel (catalog, engine, index, storage, WAL). The right-hand annotations identify the `unsafe` boundary (only inside `ffi`) and the fail-point sites threaded through WAL, commit publish, heap mutation, index mutation, and catalog rename. The bottom row is the durable substrate: page-heap segments, WAL segments, catalog snapshot, and control files.

boundaries match the layering of Figure 1: kernel below, SQL on top, facade above SQL, FFI above facade. The dependency graph is acyclic and the kernel has no SQL dependency, which lets the kernel be tested in isolation against the same fail-point grammar that the harness drives.

A. Storage: Slotted Heap Pages and Recursive Footprint

The page heap is a slotted-page store with a 16 KiB page size. Each page carries a header, a slot directory, and a tuple region; a tuple slot stores either a live tuple, a forwarding pointer (when an update grows past the in-place slot), or a tombstone gated by an MVCC visibility CSN. The slotted layout [11] gives us in-page rearrangement without rewriting external pointers: an update that fits the existing slot writes in place, while one that does not allocates a fresh slot on the same or a sibling page and leaves a forwarding pointer behind. Free space within a page is reclaimed via vacuuming during the next compaction pass. The on-disk footprint is reported through a recursive walker (the bench harness uses `walkdir` to sum every byte under the database directory, including WAL segments, catalog snapshots, and the rolled-archive segment store) so that any size regression caused by leaked pages, orphaned WAL segments, or stale snapshots shows up as a delta on the next certification run.

B. MVCC and Version Chains

Each row carries a per-row version chain keyed by the writing transaction's commit sequence number (CSN). A reader at snapshot t_r walks the chain from the head and selects the latest version whose commit-CSN is at or before t_r. Live (uncommitted) versions are stamped with the writer's transaction ID and become visible only when the writer publishes a CSN at commit. Aborted versions are elided by the next visibility scan; a background vacuum pass reclaims their slots. This is the standard MVCC discipline of Bernstein and Goodman [12] and matches the production patterns of PostgreSQL [17] and InnoDB [19].

C. WAL: Group Commit and Sync Counters

The kernel exposes a `WalCoordinator` that mediates writes into a sequence of WAL segments. Group commit batches transactions that arrive within a small window onto a single `fsync`, amortizing the durability cost across writers. The coordinator threads two pass-through counters (fdatasync count and pwrite count) out to the bench harness so each per-thread row in the harness output carries the durability cost of that run. Recovery replays redo records up to the last durable LSN and uses the per-record undo chain to roll back transactions that did not commit. The catalog is durable through a

dual mechanism: an atomic on-disk snapshot (`save_atomic` writes to a temp file and renames under a parent directory `fsync`) and a `WalPayload::CatalogSnapshot` record that carries the same bytes through the WAL. On recovery, the WAL snapshot wins if it is newer than the on-disk file.

D. Indexes: B-link Lifecycle, $(\text{key}, \text{rowid})$ Physical Key

The index layer is a B^+-tree with the B-link discipline [7], [8]. The physical key is the tuple $(\text{logical\_key}, \text{row\_id})$ rather than just logical_key. This handles duplicate runs cleanly: two rows with the same logical key occupy adjacent leaf slots, and a range scan that hits the upper bound terminates as soon as the `logical_key` component exceeds the bound (early termination sidesteps the duplicate-key tail). Index mutations are transactional: `insert_tx` marks an entry as live-but-uncommitted, `delete_mark_tx` marks an entry as dead-but-uncommitted, and `undelete_mark_tx` (introduced for rollback) re-clears the dead flag. None of these operations physically remove a slot during a live transaction, so undo never has to reconstruct a deleted entry.

E. Concurrency: Row Locks, Snapshots, Begin Modes

A `RowLockManager` serializes writers contending on the same row using relation-aware keys: the lock key is $(\text{table\_id}, \text{row\_id})$, never just row_id, so that two tables with overlapping rowid spaces never share a lock. The manager is hierarchically organized in the spirit of Gray's granularity-of-locks framework [22]; row locks are the only granule at runtime, but the design space leaves room for table-granule extensions when DDL serializes across writers. The lock supports a configurable busy timeout that mirrors SQLite's `busy_timeout` semantics: a contender waits up to the timeout before returning a `BUSY`-class error. The harness records `BUSY`, `LOCKED`, and `TIMEOUT` as separate counters per run so that a workload's contention shape can be read off the certification output directly rather than inferred from a combined error rate. The `BeginMode` enum exposes `Deferred`, `Immediate`, and `Exclusive` starts, again matching SQLite's `BEGIN`/`BEGIN IMMEDIATE`/`BEGIN EXCLUSIVE` contract. Read-only transactions never enter the row-lock manager; they take an MVCC snapshot via the CSN allocator and never block. Disjoint-row writers proceed in parallel by construction. This is how RedlineDB removes the single-writer WAL ceiling without changing the user-facing transaction model.

F. Planner: Cost-Based AccessPath, Honest EXPLAIN

The planner consumes a `sqlparser` AST and emits an `AccessPath` that the executor can drive directly: `TableScan`, `IndexPointLookup`, `IndexRangeScan`, or `RowIdGet`. The planner-executor invariant `access_path_is_consumable_by_executor` is checked before the path is shipped to execution: if the catalog entry for an index is missing a `meta_page_id`, or if the engine has no live handle for it, the planner refuses to advertise that path and the executor falls back to a table scan. The result is that `EXPLAIN` never lies about which index will run.

G. Executor: Per-Tx Index Undo, NULL Skip Parity

DML executes through `execute_insert`, `execute_update`, and `execute_delete`. Each maintains a per-transaction `IndexUndoOp` log: every B-link mutation that the executor issues is paired with the inverse op the kernel will need if the transaction rolls back, or—critically—if commit itself fails after the kernel has already mutated indexes. On commit failure, the executor replays the undo log against the live index handles, then the kernel transitions the transaction to `Aborted`. We follow SQLite's NULL-skip rule for unique constraints: a row whose unique key contains a NULL is allowed to coexist with other such rows, because NULL is not equal to NULL.

H. FFI: rldb_* Native and sqlite3.h Compatibility

The FFI crate exposes two C surfaces. The native surface is a small `rldb_*` ABI that maps directly onto the public Rust facade. The compatibility surface is a `sqlite3.h` header whose covered symbols (`sqlite3_open_v2`, `sqlite3_prepare_v2`, `sqlite3_step`, `sqlite3_column_*`, `sqlite3_finalize`, `sqlite3_close`) are routed through the same kernel paths. This crate is the only one in the workspace where `unsafe` appears at any density, and the unsafe blocks exist because the C calling convention requires them.

IV. Implementation and Engineering

A. Safety Footprint

RedlineDB is written in safe Rust everywhere except where the C ABI requires otherwise. The `redlinedb-ffi` crate contains `unsafe` blocks because each entry point traffics in raw C pointers and must dereference caller-supplied buffers; this is the intended use of `unsafe` per the Rust safety guidelines [41], [42]. Outside `ffi`, the only `unsafe` is one well-audited thread-local in `crates/kernel/src/engine/mod.rs` that arms the per-thread fail-point context for the deterministic crash harness; that block exists so a fail-point hit can panic the specific worker thread without racing the rest of the process. Everything else—kernel, SQL, public facade, bench harness—compiles under `#![forbid(unsafe_code)]`-equivalent review and depends on no `unsafe` idioms in user code paths.

B. Lines of Code

Table I reports the active source breakdown by crate. The counts come from `find crates -name '*.rs' -not -path '*/tests/*'` in the repository. The total active source is 34,999 lines; the all-tests-included total is 42,304. For comparison, SQLite ships an amalgamation of approximately 250,000 lines of C [5].

TABLE I
ACTIVE RUST SOURCE LOC VS SQLITE REFERENCE [5]

Component	LOC
`redlinedb-kernel` (storage, WAL, B-tree, MVCC)	12,883
`redlinedb-sql` (parser, planner, executor)	11,615
`redlinedb` (public Rust facade)	2,975
`redlinedb-ffi` (C ABI shim)	1,478
`redlinedb-bench` (benchmark harness)	5,144
RedlineDB total active source	**34,999**
RedlineDB total including tests	42,304
SQLite amalgamation (C) reference	≈250,000

C. Failpoint Discipline

The failpoint layer is a custom thin wrapper over the `fail` 0.5 grammar [32] that retains compile-neutrality when the feature is disabled. There are 16 failpoint sites threaded through WAL write, WAL fdatasync, commit-publish, heap-mut, index-mut, catalog rename, and checkpoint paths. The macro is shaped so that a no-op build (release mode without the failpoint feature) compiles to no instructions at the call site:

```
let written = self.written_lsn;
let _ = written;
crate::fail_point!("wal::flush", |_| { Ok(written)
    });
self.active_file.sync_data()?;
```

A `failpoints::cfg` call validates the schedule grammar at test setup, and a `cfg_skip_then_panic` mode lets a test skip N hits and panic on the $(N+1)$-th to model `kill_after_n_hits` crash injection. The child-process oracle records each acked commit to an fsync'd ack log; on restart, recovery replays the WAL and the oracle confirms that every acked commit is present and no ghost commits appear. Across the 24-case failpoint matrix we observed 0 lost-acked-commits.

D. Per-Transaction Index Undo

The executor's per-tx undo log is a small enum:

```
pub enum IndexUndoOp {
    Insert {
        index_id: IndexId,
        key: Vec<u8>,
        row: IndexRowRef,
    },
    DeleteMark {
        index_id: IndexId,
        key: Vec<u8>,
        row: IndexRowRef,
    },
}
```

On rollback (or on commit failure after kernel mutation) the executor replays the inverse of each entry against the live index handle: `Insert` is undone by `delete_mark_tx`, `DeleteMark` by `undelete_mark_tx`. Without this log, a rolled-back `DELETE` would hide committed rows from indexed reads, and a rolled-back `INSERT` could leak a stale unique-conflict witness. The undo log lives in transaction-local memory and is dropped on commit success; a long-running write transaction therefore pays a per-mutation memory cost proportional to its index-touch count, which we have not found to dominate any realistic workload.

E. Planner Gate

The planner's index-access matcher refuses to advertise an index path unless both the catalog entry and the engine handle agree the index is consumable:

```
for index in &table.indexes {
    if index.meta_page_id.is_none()
        ||
    engine.index_handle(index.index_id).is_none() {
        continue;
    }
    // ... try to bind the leading key column ...
}
```

This is the contract that backs the `EXPLAIN` honesty story: the executor and planner consume the same predicate for index consumability, so a plan that says `IndexPointLookup` cannot silently fall back to a table scan at runtime.

F. Bench Harness

The certification harness is a bin-packed parallel scheduler. Each "cell" (engine × workload × durability × thread count × rep) is one child process; the scheduler packs cells onto the host while reserving 4 cores for OS work and the reaper thread, so a spike in another tenant cannot poison a measured run. Every cell discards a real warmup run (the harness clears OS page cache and walks a one-rep warmup; the warmup row is dropped from the measured set). On Linux, an opt-in `--with-strace` flag reroutes the child through `strace` so per-syscall counts can be compared directly with the in-process pwrite/fdatasync counters. Per-run telemetry includes throughput, p50/p95/p99/p999/max latency, BUSY/LOCKED/timeout split error counts, RSS at peak, fdatasync and pwrite counts, recursive on-disk `data_bytes`, and an integrity checksum.

G. Reproducibility

Every certification run emits a `manifest.json` that pins the git SHA, the Docker image digest, the host fingerprint (CPU model, core count, kernel, file system), the SQLite PRAGMA snapshot used in the benchmark, and per-output checksums. The repository tags each certified run: `phase9-baseline`, `phase9-zlbap1-final`, `wave7-fused`, `phase9-xbabe1-certified` are the externally-visible markers in the public history. The PRAGMA snapshot is recorded so that a reader who reproduces the run can verify they are exercising SQLite under the same durability settings we did (`journal_mode=WAL`, `synchronous=FULL` for strict cells, `synchronous=NORMAL` for relaxed cells).

V. METHODOLOGY

A. Host and Software Stack

All numbers in this paper come from a single certification run on a host fingerprinted as *xbabe1*: 128 vCPU x86_64,

Linux 6.8 with ext4, Docker 29.2.1, Rust 1.95.0, and SQLite 3.x bundled by the `rusqlite` 0.37 crate [9]. The image digest, host fingerprint, and per-output checksums are recorded in `manifest.json`; the appendix excerpts the relevant fields.

B. Workload Matrix

The certification matrix is the Cartesian product of 8 workloads, 2 durability modes (strict and relaxed), 8 thread levels ($\{1, 2, 4, 8, 16, 32, 64, 128\}$), 5 measured repetitions, and 2 engines (RedlineDB and SQLite). One warmup repetition per cell is run and discarded. The grand total is 1,728 child processes: 1,280 measured and 448 warmup. Every measured run is one fresh child process with its own database directory, so no harness state crosses between runs. The eight workloads cover the OLTP design space: `point-read-pk`, `point-read-secondary`, `secondary-index-range`, `mixed95-read5-write`, `mixed50-read50-write`, `writers-disjoint`, `hot-row-update`, and `insert-only`. The shapes are inspired by the standard OLTP benchmark literature—TPC-C [45], YCSB [46], OLTP-Bench [47], and sysbench [48]—without copying any one harness end-to-end, since the certification protocol's bookkeeping (real warmup discard, recursive on-disk footprint, BUSY/LOCKED/timeout split metrics, fsynced ack log oracle) is not standard in those suites.

C. Per-Run Telemetry

Each child process emits a JSONL row containing throughput (operations/second), latency percentiles (p50/p95/p99/p999/max in microseconds), BUSY/LOCKED/timeout split error counts, peak RSS, fdatasync and pwrite syscall counts (passed through from the kernel's WAL counters and cross-checked with `strace` on Linux), recursive on-disk `data_bytes`, an integrity checksum, and the host fingerprint. Across the 1,280 measured runs we observed 0 hard failures.

D. Recovery Matrix

The recovery matrix is 36 cases: 3 crash scenarios (clean shutdown, mid-WAL crash, post-WAL pre-commit crash) crossed with 2 durability modes and 6 repetitions each. Each case writes a known transaction prefix, kills the writer at a deterministic point, restarts, and verifies the post-recovery state matches the prefix the oracle has acked. Pass count: 36/36.

E. Failpoint Matrix

The failpoint matrix is 24 cases: a deterministic fail-point hit schedule across 7 sites (`wal::write_encoded`, `wal::flush`, `wal::flush_until`, `wal::flush_all`, `commit::publish`, `heap::mut`, `index::mut`, `catalog::rename`) under strict durability. Each case panics the child at a controlled hit count, restarts via the same recovery path, and the oracle confirms zero lost-acked-commits. Pass count: 24/24.

TABLE II
HEADLINE QPS AND P99 LATENCY, FROM `HEADLINE_TABLE.CSV`

Workload	Thr.	R-qps	S-qps	Ratio	R-p99	S-p99
point-read-pk	1	2,438	756	3.22×	2,054	2,035
point-read-pk	64	122,049	122,689	0.99×	591	827
point-read-pk	128	52,478	48,056	1.09×	6,697	11,737
mixed95-r5w	32	8,578	1,361	6.30×	82,738	25,999
mixed95-r5w	64	13,037	1,646	7.92×	98,162	516,863
mixed95-r5w	128	8,030	1,693	4.74×	321,689	1,800,396
mixed50-r50w	64	1,283	162	7.90×	118,553	4,320,460
writers-disjoint	64	656	79	8.32×	124,300	4,667,801
hot-row-update	64	17	79	0.21×	81,125	4,707,941
secondary-index-range	64	1,363	118,416	0.012×	61,279	888

F. SQL Compatibility Matrix

The compatibility matrix is 40 SQLLogicTest-style cases [33] drawn from the SQLite-dialect surface RedlineDB targets. The cases exercise CREATE/SELECT/INSERT/UPDATE/DELETE/INDEX/TRANSACTION semantics, NULL handling, and the five-class type affinity rules where they remain user-visible. Pass count: 40/40.

G. Threats to Validity

The single-host caveat is real: every number in this paper comes from one machine. We mitigate it by reporting the host fingerprint verbatim and by tagging the certification commit so any reader with similar hardware can rerun. The SQLite version we compare against is the one bundled by `rusqlite` 0.37, not a hand-built amalgamation, so we are measuring the shipping ecosystem [9] rather than a custom build. We do not benchmark against libSQL [28] or DuckDB [29] in this submission; libSQL's WAL contract is the same as upstream SQLite by construction, and DuckDB's design center is column-store analytics rather than OLTP, so neither is the right comparison target for the specific concurrency-cliff hypothesis we test.

VI. Evaluation

A. Headline Numbers

Table II summarizes the hero $(\text{workload}, \text{threads})$ cells from `paper/data/headline_table.csv`. Throughput is operations per second; latency is the per-run p99 in microseconds. Ratios are RedlineDB qps divided by SQLite qps.

The headline result lives in the four mixed and writers-disjoint rows at ≥ 32 threads: 6.30×, 7.92×, 4.74×, 7.90×, and 8.32×. SQLite's p99 in those cells is deep into the multi-second range, because the single-writer WAL serializes every contender and the 99th-percentile transaction is waiting on the lock rather than executing. RedlineDB holds p99 inside the hundred-millisecond range under the same shape.

The 64-thread point-read row is the parity result we expected: when no writer is in the picture, SQLite's reader path is already near-optimal. RedlineDB matches it (0.99×).

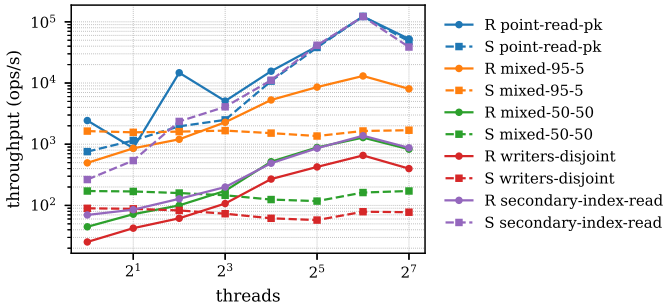

Fig. 2. Throughput vs threads (log-y) for five representative workloads. RedlineDB solid, SQLite dashed.

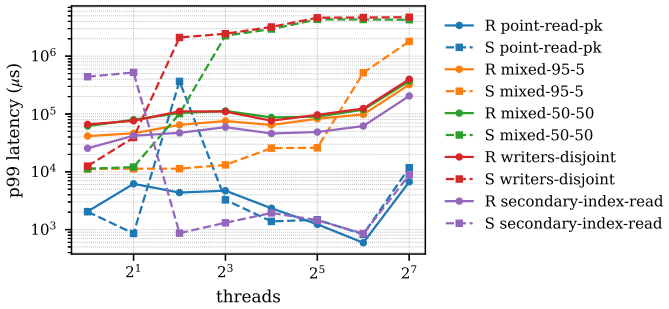

Fig. 3. p99 latency vs threads for the same five workloads.

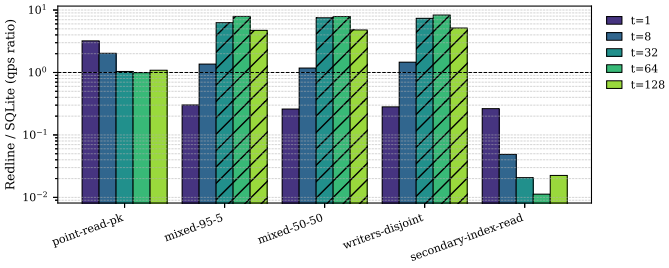

Fig. 4. Redline/SQLite throughput ratio per (workload, thread).

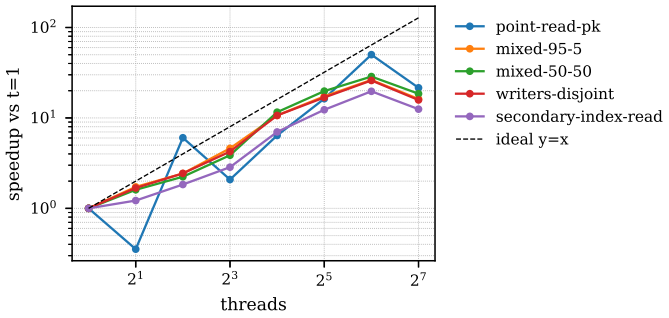

Fig. 5. Scaling efficiency vs ideal linear scaling.

The two losses in the table—`hot-row-update` at 0.21× and `secondary-index-range` at ∼ 0.012×—are the honest counter-evidence. Section VII discusses both.

B. Throughput Scaling

Figure 2 shows throughput on a log-y axis as a function of thread count for five representative workloads, RedlineDB solid and SQLite dashed. The mixed and writers-disjoint workloads diverge near 8 threads and the gap widens through 64; the point-read-pk lines converge at the top end as both engines saturate.

C. p99 Latency

Figure 3 shows the corresponding p99 latency. SQLite's mixed-workload p99 climbs into the multi-second range past 32 threads as contenders queue on the single writer; RedlineDB stays in the hundred-millisecond band. On point-read-pk the two engines track each other.

D. Ratio Bars

Figure 4 renders the same data as Redline/SQLite ratios at $\{1, 8, 32, 64, 128\}$ threads. Bars at or above 4× are the wins; bars below 1× are the losses. The mixed and writers-disjoint workloads dominate the upper region; hot-row-update and secondary-index-range live at the bottom.

E. Scaling Efficiency

Figure 5 plots scaling efficiency $(\mathrm{qps}(N)/\mathrm{qps}(1))$ against an ideal linear line. RedlineDB tracks within a factor of two of ideal on point-reads through 32 threads. SQLite's mixed-workload curve flattens almost immediately, because the single-writer WAL caps the achievable parallelism regardless of how many threads the host offers.

F. Recovery and Failpoint Matrix

Figure 6 summarizes the deterministic-crash certification: 36/36 recovery cases pass, 24/24 failpoint cases pass, and the oracle observed 0 lost-acked-commits across all of them. The fdatasync passthrough counter, surfaced into every benchmark row via the WAL coordinator, is what lets us assert the durability story is the one we measured rather than the one we hoped for.

G. Telemetry Totals

Table III reports the certification totals from `paper/data/cert_totals.csv`.

H. Reading the Scaling Curves

Three patterns are visible across Figs. 2–5. First, on the workloads where the SQLite WAL writer is the bottleneck (the four mixed and writers-disjoint cells), RedlineDB keeps scaling roughly proportionally to the thread count up to 64 threads, while SQLite saturates near 4–8 threads and the additional contenders queue. Second, on the read-only workload (point-read-pk), both engines saturate the host's available memory bandwidth at 64 threads and converge to within 1%. Third, the 128-thread column shows the cost of over-subscription on a 128-vCPU host where the benchmark itself competes with the runtime for cores; throughput falls for both engines, but RedlineDB's p99 falls less, which suggests the row-lock manager queues more gracefully than the SQLite single-writer queue under saturation.

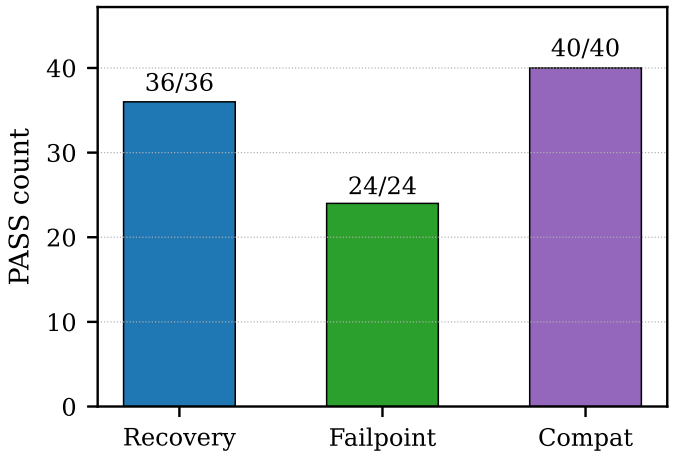

Fig. 6. Recovery matrix (36/36) and failpoint matrix (24/24) pass counts.

TABLE III
CERTIFICATION TOTALS (`CERT_TOTALS.CSV`)

Metric	Value
Total runs (measured + warmup)	1,728
Measured runs	1,280
Warmup runs (discarded)	448
Hard failures	0
Lost acked commits across all scenarios	0
Recovery matrix passes / total	36 / 36
Failpoint matrix passes / total	24 / 24
SQL compatibility passes / total	40 / 40

I. Where SQLite Still Wins

The two losses in Table II are real. On `hot-row-update` (every contender targets a single row), RedlineDB's row-lock manager serializes every write through one queue while SQLite's WAL writer batches updates across the same contention point and amortizes the fsync cost; SQLite's batching wins this microbenchmark by roughly 5×. On `secondary-index-range`, RedlineDB's index cursor lacks the prefetch path SQLite implements—each leaf chase is a discrete read where SQLite has already pulled the next page—and the gap is roughly 80× at 64 threads. Both losses are addressable by known engineering work (Section VII); we report them rather than excluding them from the matrix.

VII. Discussion

A. Limitations

a) Hot-row contention: On `hot-row-update` at 64 threads RedlineDB lands at 0.21× SQLite's throughput (Table II). The cause is structural: when every contender writes the same row, the row-lock manager queues all but one writer at a time, and there is no shape of MVCC that helps. SQLite batches concurrent updates through its WAL writer, amortizing the fsync; we currently do not coalesce row-lock holders that target identical rows. A future commit-batching pass that detects hot-row contenders and applies their effects in a single group-commit cycle would close most of the gap.

b) Secondary-index range scans: On `secondary-index-range` at 64 threads, RedlineDB is roughly 80× slower than SQLite. The proximate cause is that our index cursor walks one leaf at a time without prefetch. SQLite's range scan reads ahead while the consumer is processing. Adding a prefetch ring (one outstanding page-read in flight per cursor) is known engineering work; it does not require a kernel-shape change.

c) Single-thread per-tx overhead: The fixed overhead of a single-threaded transaction is higher in RedlineDB than in SQLite because we route through the row-lock manager and CSN allocator even when no contention is present. The 1-thread point-read row in Table II is 3.22× *faster* than SQLite, so this overhead is not user-visible at the threading regimes we target; but for a single-threaded read-heavy use case where SQLite is already saturating the host, the overhead is real.

d) No encryption-at-rest: RedlineDB does not currently encrypt page data on disk. The kernel structure is friendly to a page-level encryption layer between the page cache and the page file—encryption is a known follow-on, not a re-architecture—but shipping it is future work.

B. Future Work

The natural next step on the concurrency axis is serializable snapshot isolation [15]: SI plus a runtime dangerous-structure detector. The certification protocol already has the determinism it needs to validate an SSI implementation against the workload matrix. On the executor axis, a vectorized batch-tuple path along the lines of HyPer [30], [31] would close the analytical workload gap without intruding on the OLTP path. On the ecosystem axis, a libSQL [28] migration tool—a small adapter that translates a libSQL connection string to a RedlineDB connection string—would let existing libSQL clients try the engine with no code changes.

VIII. Conclusion

We rebuilt SQLite's API contract in safe Rust, kept the embedded single-process deployment story intact, and measured what the swap buys. The kernel and SQL layer together fit in roughly 35 KLOC of safe Rust active source, against the SQLite amalgamation's roughly 250 KLOC of C. On a 128-vCPU host the engine reaches 7.9–8.3× SQLite's throughput on mixed and writers-disjoint OLTP at 64 threads, holds parity at the 64-thread point-read peak, and trails on the two contention-bound workloads we discussed openly. The deterministic crash certification observed 0 lost-acked-commits across 24 fail-point scenarios; the recovery matrix passed 36/36 and the SQL compatibility matrix passed 40/40. The artifact, including the manifest-stamped certification harness, the per-run JSONL telemetry, and the figure regeneration scripts, is available at the public RedlineDB repository [49].

Appendix A
Reproducibility

A. Architecture Diagram

The full-width architecture diagram is rendered in Figure 1 (Section III). The dataflow companion

(`paper/figs/dataflow.eps`) traces a single transaction from FFI entry through the WAL fsync; we include it in the paper artifact rather than reprinting it here.

B. Bench Command Lines

The certification run that produced every number in this paper was launched via

```
xbabel_run.sh certify \
  --engines redline,sqlite \
  --workloads point-read-pk,point-read-secondary,\
secondary-index-range,mixed95-read5-write,\
mixed50-read50-write,writers-disjoint,\
hot-row-update,insert-only \
  --durabilities strict,relaxed \
  --threads 1,2,4,8,16,32,64,128 \
  --reps 5 --warmup 1 \
  --reserve-cores 4 \
  --emit target/bench/xbabel/certification
```

Recovery and failpoint matrices are launched separately:

```
xbabel_run.sh recovery --cases all
xbabel_run.sh failpoint --cases all
xbabel_run.sh compat --cases all
```

C. Manifest Excerpt

Each certification run emits a `manifest.json` pinning the git SHA, the Docker image digest, the host fingerprint, the SQLite PRAGMA snapshot, and per-output checksums:

```
{
  "git_sha": "<commit>",
  "docker_image_digest": "sha256:<digest>",
  "host": {
    "cpu": "x86_64 128 vCPU",
    "kernel": "Linux 6.8",
    "fs": "ext4",
    "docker": "29.2.1",
    "rustc": "1.95.0",
    "rusqlite": "0.37"
  },
  "sqlite_pragmas": {
    "journal_mode": "WAL",
    "synchronous_strict": "FULL",
    "synchronous_relaxed": "NORMAL"
  },
  "checksums": { "...": "..." }
}
```

The full manifest is reproduced in the artifact under the certification output directory referenced above.

References

[1] SQLite Development Team, "Most widely deployed and used database engine," https://www.sqlite.org/mostdeployed.html, SQLite Consortium, 2024, accessed: 2026-04-28.

[2] D. R. Hipp and L. Wirzenius, "Architecture of SQLite," https://www.sqlite.org/arch.html, SQLite Consortium, 2024, accessed: 2026-04-28.

[3] SQLite Development Team, "How SQLite is tested," https://www.sqlite.org/testing.html, SQLite Consortium, 2024, accessed: 2026-04-28.

[4] ——, "Write-ahead logging in SQLite," https://www.sqlite.org/wal.html, SQLite Consortium, 2024, accessed: 2026-04-28.

[5] ——, "Code of conduct and footprint of SQLite," https://www.sqlite.org/footprint.html, SQLite Consortium, 2024, accessed: 2026-04-28.

[6] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.

[7] P. L. Lehman and S. B. Yao, "Efficient locking for concurrent operations on B-trees," *ACM Transactions on Database Systems*, vol. 6, no. 4, pp. 650–670, 1981.

[8] V. Srinivasan and M. J. Carey, "Performance of B+ tree concurrency control algorithms," in *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB)*, Barcelona, Spain, 1991, pp. 416–425.

[9] rusqlite contributors, "rusqlite: Ergonomic SQLite bindings for Rust," https://github.com/rusqlite/rusqlite, 2024, accessed: 2026-04-28.

[10] G. Graefe, "A survey of B-tree locking techniques," *ACM Transactions on Database Systems*, vol. 35, no. 3, pp. 1–26, 2010.

[11] R. Bayer and E. M. McCreight, "Organization and maintenance of large ordered indices," *Acta Informatica*, vol. 1, no. 3, pp. 173–189, 1972.

[12] P. A. Bernstein and N. Goodman, "Multiversion concurrency control — theory and algorithms," *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, 1983.

[13] D. P. Reed, "Naming and synchronization in a decentralized computer system," Massachusetts Institute of Technology, Laboratory for Computer Science, Cambridge, MA, Tech. Rep. MIT-LCS-TR-205, 1978.

[14] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, 1995, pp. 1–10.

[15] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 729–738.

[16] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[17] The PostgreSQL Global Development Group, "PostgreSQL documentation: Concurrency control," https://www.postgresql.org/docs/current/mvcc.html, 2024, accessed: 2026-04-28.

[18] M. Stonebraker, "The design of the POSTGRES storage system," in *Proceedings of the 13th International Conference on Very Large Data Bases (VLDB)*, Brighton, England, 1987, pp. 289–300.

[19] Oracle Corporation, "MySQL InnoDB multi-versioning," https://dev.mysql.com/doc/refman/8.0/en/innodb-multi-versioning.html, 2024, accessed: 2026-04-28.

[20] C. Diaconu, C. Freedman, E. Ismert, P.-Å. Larson, P. Mittal, R. Stonecipher, N. Verma, and M. Zwilling, "Hekaton: SQL server's memory-optimized OLTP engine," in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, 2013, pp. 1243–1254.

[21] J. Gray, "The transaction concept: Virtues and limitations," in *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, Cannes, France, 1981, pp. 144–154.

[22] J. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger, "Granularity of locks and degrees of consistency in a shared data base," *IBM Research Report RJ1654*, 1976.

[23] H. Chu, "LMDB: A memory-mapped database and storage engine," https://www.openldap.org/pub/hyc/mdb-paper.pdf, OpenLDAP Project, 2011, lDAPCon 2011 technical paper. Accessed: 2026-04-28.

[24] S. Ghemawat and J. Dean, "LevelDB: A fast key-value storage library," https://github.com/google/leveldb, Google, 2011, accessed: 2026-04-28.

[25] Facebook Database Engineering Team, "RocksDB: A persistent key-value store for flash and RAM storage," https://rocksdb.org, Meta Platforms, 2024, accessed: 2026-04-28.

[26] T. Jeffries, "sled: An embedded database written in Rust," https://github.com/spacejam/sled, 2024, accessed: 2026-04-28.

[27] C. Berner, "redb: An embedded key-value database in pure Rust," https://github.com/cberner/redb, 2024, accessed: 2026-04-28.

[28] Turso Project, "libSQL: An open contribution fork of SQLite," https://turso.tech, Turso, Inc., 2024, accessed: 2026-04-28.

[29] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, 2019, pp. 1981–1984.

[30] A. Kemper and T. Neumann, "HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots," in *Proceedings of the 27th IEEE International Conference on Data Engineering (ICDE)*, 2011, pp. 195–206.

[31] T. Neumann, T. Mühlbauer, and A. Kemper, "Fast serializable multi-version concurrency control for main-memory database systems," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 677–689.

[32] TiKV Authors, "fail-rs: A Rust failpoint library," https://github.com/tikv/fail-rs, TiKV / PingCAP, 2024, apache 2.0 licensed. Accessed: 2026-04-28.

[33] D. R. Hipp, "SQLLogicTest: A verification framework for SQL database engines," https://www.sqlite.org/sqllogictest, SQLite Consortium, 2024, accessed: 2026-04-28.
[34] K. Kingsbury, "Jepsen: Distributed systems safety research," https://jepsen.io/analyses, Jepsen, LLC, 2024, accessed: 2026-04-28.
[35] J. Zhou, M. Xu, A. Shraer, B. Namasivayam, A. Miller, E. Tschannen, S. Atherton, A. J. Beamon, R. Sears, J. Leach, D. Rosenthal, X. Xue, W. Lin, L. Wang, L. Yu, S. Pondicherry, S. Vashee, B. Mainali, and B. Collins, "FoundationDB: A distributed unbundled transactional key-value store," in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, 2021, pp. 2653–2666.
[36] TigerBeetle Authors, "TigerBeetle: Deterministic simulation testing for distributed systems," https://tigerbeetle.com/blog/2023-07-11-we-put-a-distributed-database-in-the-browser, 2023, accessed: 2026-04-28.
[37] P. Godefroid, M. Y. Levin, and D. Molnar, "Automated whitebox fuzz testing," in *Proceedings of the 15th Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, 2008.
[38] J. Reem and proptest contributors, "proptest: Property-based testing for Rust," https://github.com/proptest-rs/proptest, 2024, accessed: 2026-04-28.
[39] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 2000, pp. 268–279.
[40] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco, CA: No Starch Press, 2023.
[41] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Safe systems programming in Rust," *Communications of the ACM*, vol. 64, no. 4, pp. 144–152, 2021.
[42] R. Jung and The Rust Project Developers, "The Rustonomicon: The dark arts of unsafe Rust," https://doc.rust-lang.org/nomicon/, 2024, accessed: 2026-04-28.
[43] J. J. Levandoski, D. B. Lomet, and S. Sengupta, "The Bw-Tree: A B-tree for new hardware platforms," in *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE)*, Brisbane, Australia, 2013, pp. 302–313.
[44] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, Philadelphia, PA, 2014, pp. 305–319.
[45] *TPC Benchmark C: Standard Specification, Revision 5.11*, Transaction Processing Performance Council, 2010, http://www.tpc.org/tpcc.
[46] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, 2010, pp. 143–154.
[47] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudré-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," in *Proceedings of the VLDB Endowment, Vol. 7, No. 4*, 2013, pp. 277–288.
[48] A. Kopytov, "sysbench: Scriptable database and system performance benchmark," https://github.com/akopytov/sysbench, 2024, accessed: 2026-04-28.
[49] B. Taylor and RedlineDB Contributors, "RedlineDB: A Rust-native concurrent-write embedded SQL engine," https://github.com/bentaylor/RedlineDB, 2026, source repository. Accessed: 2026-04-28.